

Sri Lanka Institute of Information Technology



SE3090 – Software Engineering Frameworks
Year 3, Semester 1 - 2026

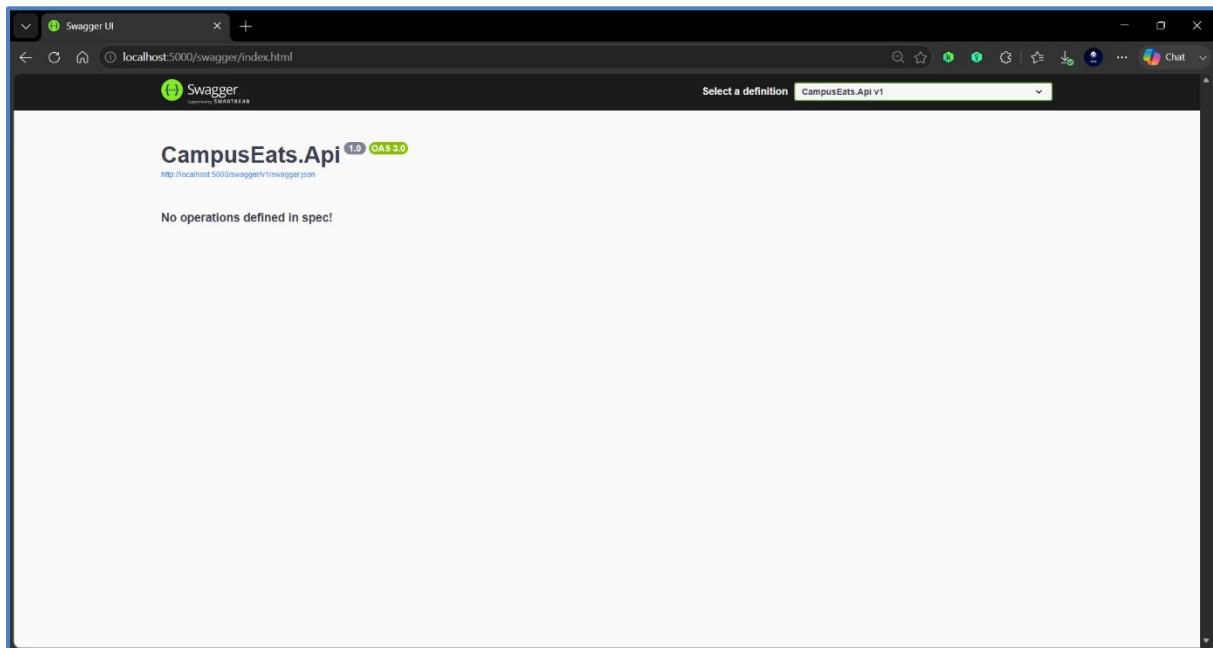
Practical Answer Submission

Practical Sheet No: 03

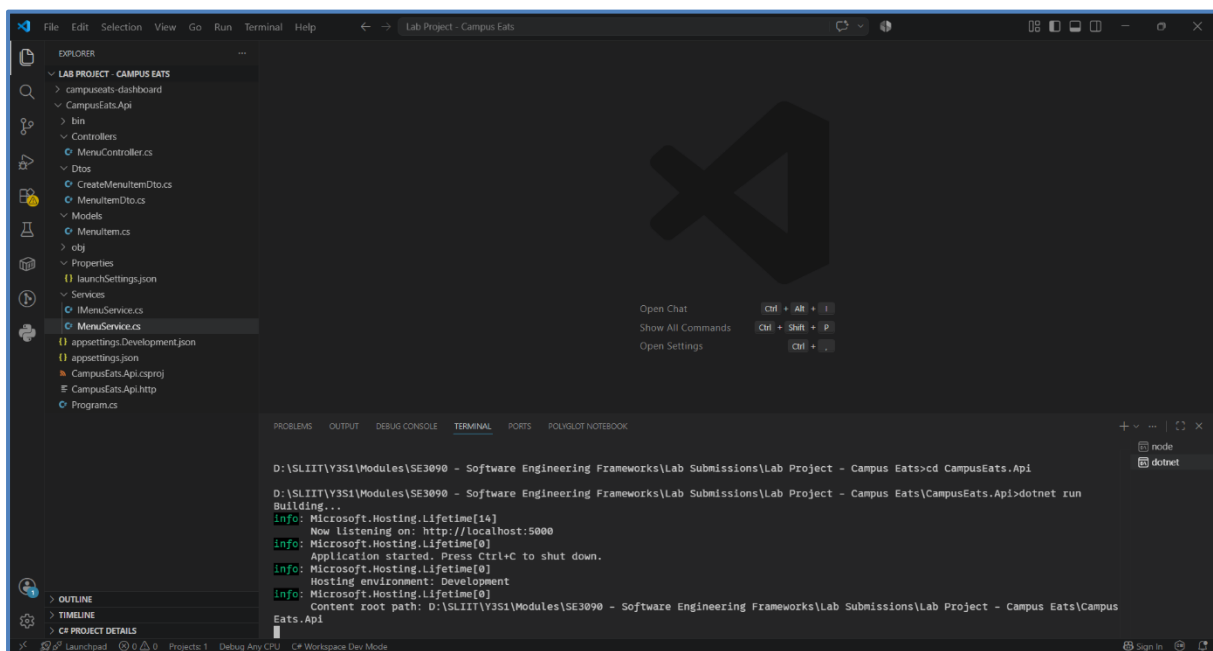
Student ID	IT24102798
Student Name	Sooriyabandara U.R.G.W.K.
Campus/ Center name	Malabe
Specialization	Artificial Intelligence
Batch No.	01.01

Task 01 – Create the Web API Project & Run Swagger

- Screenshot of the Running Swagger Page



- Screenshot of the Project's Folder Structure



Task 02 – Create the Model & DTOs

Short Explanation of Model-vs-DTO

- **Model** represents the application's internal data structure and is used by the business logic and service layer. A **DTO (Data Transfer Object)** defines the data exchanged between the API and its clients, keeping the internal model separate from the public API contract for better security, maintainability, and flexibility.

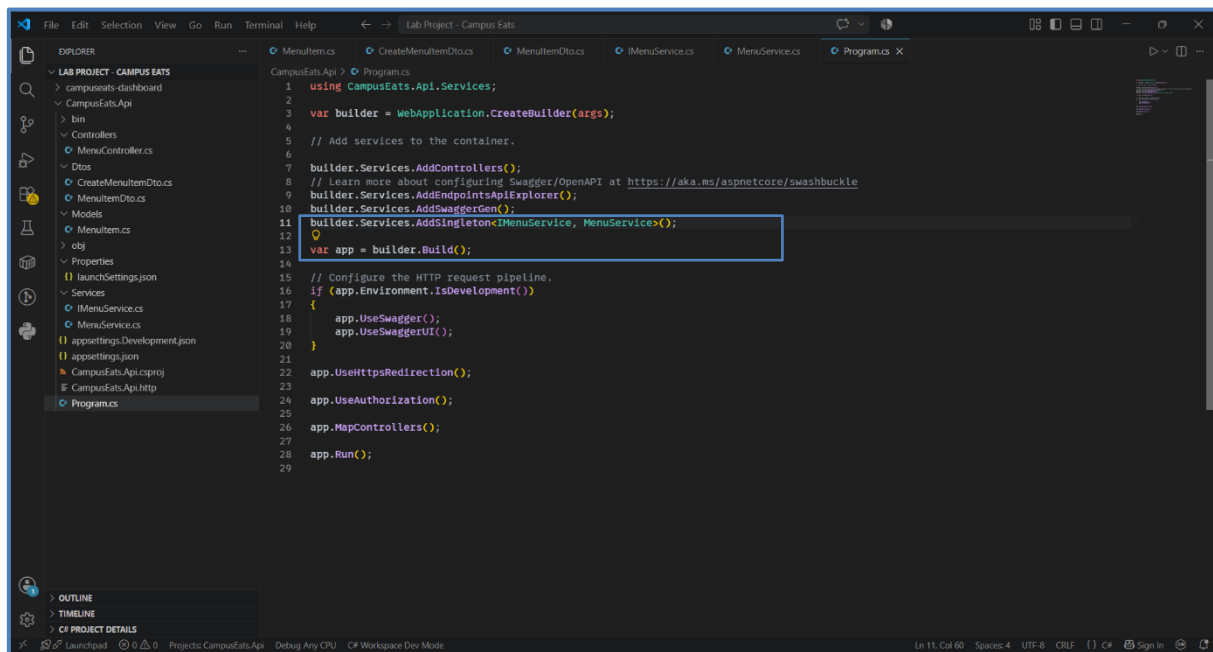
Model	DTO
• Internal representation of data.	• Public representation sent over the API.
• Used by services and business logic.	• Used for requests and responses.
• Can contain internal properties.	• Contains only the data the client should see.
• May change with application logic.	• Remains a stable API contract.

Reason why the input DTO omits “*Id*”

- The input DTO does not include an ***Id*** because the server is responsible for generating and assigning a unique identifier when a new menu item is created. This prevents clients from supplying or modifying resource IDs and ensures consistent data management.

Task 03 – Create the Service & Register it for DI

- Screenshot of the Registration Line in Program.cs



The Brief Note on the Lifetime Choice

- “AddSingleton” is used because the “MenuService” stores menu items in an in-memory list that must persist throughout the application's lifetime. Using a singleton ensures the same service instance is shared across all requests, allowing the seeded data and any changes to remain available until the application stops. With a database-backed application, “AddScoped” would typically be used instead.

Task 04 – Controller with GET + POST (Tested in Swagger)

- Screenshot of the GET Request for All Dishes
(Status Code: 200)

The screenshot displays the Swagger UI for the CampusEats.Api v1. The selected definition is 'CampusEats.Api v1'. The 'Menu' endpoint is highlighted, showing a GET request to '/api/Menu'. The 'Parameters' section indicates 'No parameters'. The 'Execute' button is visible. The 'Responses' section shows a 200 status code with a JSON response body. The response body contains an array of three menu items: 'Lotto Batti', 'Fried Rice', and 'Mataloposo'. The 'Response headers' section shows 'Content-type: application/json; charset=utf-8', 'date: Sun, 02 Aug 2020 16:56:46 GMT', 'server: Restful', and 'Transfer-Encoding: chunked'. The 'Responses' table shows a 200 status code with a description of 'OK'. The 'Media type' is set to 'text/plain'. The 'Example Value' section shows a JSON object with the same structure as the response body. The 'Schemas' section shows 'CreateMenuItemDto' and 'MenuItemDto'.

Swagger UI interface showing the GET request for the /api/Menu endpoint. The response status is 200, and the response body is a JSON array of menu items.

Menu

GET /api/Menu

Parameters

No parameters

Execute Clear

Responses

Code Details

200

Response body

```
{
  "id": 1,
  "name": "Lotto Batti",
  "price": 10,
  "category": "Main",
  "available": true
},
{
  "id": 2,
  "name": "Fried Rice",
  "price": 10,
  "category": "Main",
  "available": true
},
{
  "id": 3,
  "name": "Mataloposo",
  "price": 10,
  "category": "Dessert",
  "available": true
}
```

Response headers

```
Content-type: application/json; charset=utf-8
date: Sun, 02 Aug 2020 16:56:46 GMT
server: Restful
Transfer-Encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type: text/plain

Content Accept header:

Example Value Schema

```
{
  "id": 1,
  "name": "string",
  "price": 10,
  "category": "string",
  "available": true
}
```

Schemas

- CreateMenuItemDto >
- MenuItemDto >

- **Screenshot of the GET Request for a Missing ID with ID as 99 (Status Code: 404)**

The screenshot displays the Swagger UI for the CampusEats.Api v1. The selected definition is 'CampusEats.Api v1'. The 'Menu' section is expanded, showing three endpoints: GET /api/Menu, POST /api/Menu, and GET /api/Menu/{id}. The GET /api/Menu/{id} endpoint is selected, and the 'Parameters' tab is active. The 'id' parameter is required, of type integer (int32), and has a value of 99 entered. The 'Execute' button is clicked, and the 'Responses' section is expanded. The response for the 404 status code is shown, indicating 'Error: Not Found'. The response body is a JSON object with the following structure:

```
{  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.3",  "title": "Not Found",  "status": 404,  "traceId": "90-9ef79b4efc8c2d94b12745ab6d79cd-6d832705aca72c5f-00"}
```

The response headers are:

```
Content-Type: application/problem+json; charset=utf-8
Date: Sun, 02 Aug 2020 17:09:30 GMT
Server: Kestrel
Transfer-Encoding: chunked
```

The 'Responses' section shows a table with the following data:

Code	Description	Links
200	OK	No links

The 'Schemas' section is also expanded, showing the 'CreateMenuItemDto' and 'MenuItemDto' schemas.

- Screenshot of the POST Request for Creating a New Dish (Status Code: 201)

The screenshot displays the Swagger UI for the CampusEats.Api v1. The interface is in a dark theme. The top navigation bar shows the Swagger logo and the API definition selected. The main content area is titled "Menu" and lists two endpoints: GET /api/Menu and POST /api/Menu. The POST endpoint is selected, and its details are shown. The "Parameters" tab is active, showing no parameters. The "Request body" tab is also active, showing a JSON object: {"name": "Iced Miso", "price": 300, "category": "Drinks"}. The "Execute" button is visible. Below the request body, the "Responses" section shows the response for the POST request. The response is a 201 status code, indicating a successful creation. The response body is a JSON object: {"id": 1, "name": "Iced Miso", "price": 300, "category": "Drinks", "available": true}. The response headers are also shown: content-type: application/json; charset=utf-8, date: Sun, 02 Aug 2020 17:18:45 GMT, location: http://localhost:5000/api/Menu/4, server: Kestrel, and transfer-encoding: chunked. Below the response details, there is a table of responses with columns for Code, Description, and Links. The table shows a 200 OK response with a media type of text/plain. At the bottom, the "Schemas" section is visible, showing the CreateMenuItemDto and MenuItemDto schemas.

Swagger UI
localhost:5000/swagger/index.html
Select a definition CampusEats.Api v1

CampusEats.Api ^{1.0} OA5.3.0

http://localhost:5000/swagger/v1/swagger.json

Menu

GET /api/Menu

POST /api/Menu

Parameters

No parameters

Request body

application/json

```
{
  "name": "Iced Miso",
  "price": 300,
  "category": "Drinks"
}
```

Execute Clear

Responses

Curl

```
curl -X POST \
  http://localhost:5000/api/Menu \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Iced Miso",
    "price": 300,
    "category": "Drinks"
  }'
```

Request URL

http://localhost:5000/api/Menu

Server response

Code Details

201 Uncommented

Response body

```
{
  "id": 1,
  "name": "Iced Miso",
  "price": 300,
  "category": "Drinks",
  "available": true
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 02 Aug 2020 17:18:45 GMT
location: http://localhost:5000/api/Menu/4
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type

text/plain

Controls Accept header

Example Value Schema

```
{
  "id": 1,
  "name": "Iced Miso",
  "price": 300,
  "category": "Drinks",
  "available": true
}
```

GET /api/Menu/{id}

Schemas

CreateMenuItemDto >

MenuItemDto >

Task 05 – Add PUT + DELETE, Test Full CRUD & Justify (LO4)

Part A – Add PUT and DELETE

- Screenshot of the POST Request for Creating a New Dish
(Status Code: 201)

The screenshot displays the Swagger UI interface for the 'CampusEats.Api v1' API. The 'Menu' section is expanded, showing the 'POST /api/Menu' endpoint. The 'Parameters' tab is active, indicating no parameters are present. The 'Request body' is set to 'application/json' and contains a JSON object:

```
{  "name": "Iced Miso",  "price": 300,  "category": "Drinks"}
```

. The 'Execute' button is visible. Below the request, the 'Responses' section shows a '201' status code with a 'Response body' containing a JSON object:

```
{  "id": 0,  "name": "Iced Miso",  "price": 300,  "category": "Drinks",  "available": true}
```

. The 'Response headers' section shows:

```
content-type: application/json; charset=utf-8  date: Sun, 02 Aug 2020 17:18:45 GMT  location: http://localhost:5000/api/Menu/4  server: Kestrel  transfer-encoding: chunked
```

. The 'Responses' table shows a '200' status code with a description of 'OK' and a 'Media type' of 'text/plain'. The 'Schemas' section at the bottom lists 'CreateMenuItemDto' and 'MenuItemDto'.

- **Screenshot of the PUT Request of Changing the Data of the already Created Dish (Status Code: 204)**

The screenshot displays the Swagger UI for the CampusEats.Api v1. The 'Menu' section is expanded, showing the PUT endpoint `/api/Menu/{id}`. The 'id' parameter is required and is set to 4. The request body is a JSON object: `{ "name": "Chocolate Milkshake", "price": 450, "category": "Drinks" }`. The 'Execute' button is highlighted. Below the request, the 'Responses' section shows a 204 status code with the message 'No content'. The 'Server response' section shows the response headers: `date: Sun, 02 Aug 2020 18:16:12 GMT` and `server: Kestrel`. The 'Schemas' section at the bottom lists `CreateMenuitemDto` and `MenuitemDto`.

Swagger UI for CampusEats.Api v1. The selected definition is CampusEats.Api v1.

Menu

- GET `/api/Menu`
- POST `/api/Menu`
- GET `/api/Menu/{id}`
- PUT `/api/Menu/{id}`**

Parameters

Name	Description
id * required	Integer(Sint32) (path)

Request body

application/json

```
{  "name": "Chocolate Milkshake",  "price": 450,  "category": "Drinks"}
```

Execute Clear

Responses

Curl

```
curl -X 'PUT' \  "http://localhost:5000/api/Menu/4" \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "name": "Chocolate Milkshake",  "price": 450,  "category": "Drinks"  }'
```

Request URL

`http://localhost:5000/api/Menu/4`

Server response

Code	Details
204	undocumented

Response headers

```
date: Sun, 02 Aug 2020 18:16:12 GMT  server: Kestrel
```

Responses

Code	Description	Links
200	OK	No links

Schemas

- CreateMenuitemDto >
- MenuitemDto >

- **Screenshot of the DELETE Request for the already Updated Dish (Status Code: 204)**

The screenshot displays the Swagger UI interface for the 'CampusEats.Api v1' API. The 'Menu' section is expanded, showing a list of endpoints. The 'DELETE /api/Menu/{id}' endpoint is selected, highlighted in red. Below the endpoint list, the 'Parameters' section is visible, showing a required integer parameter 'id' with a value of '4'. The 'Execute' button is present. The 'Responses' section shows a 204 status code response with headers: 'date: Sun, 02 Aug 2026 18:18:35 GMT' and 'server: RestErl'. The 'Schemas' section at the bottom lists 'CreateMenuItemDto' and 'MenuItemDto'.

Swagger UI interface showing the DELETE request for the already Updated Dish (Status Code: 204).

The interface displays the API definition for **CampusEats.Api v1** (OAS 3.0) at <http://localhost:5000/swagger/v1/swagger.json>.

The selected endpoint is **DELETE /api/Menu/{id}**.

Parameters:

Name	Description
id * required	Integer (Sint32) (note)

Input value: 4

Responses:

204 no content

Response headers:

```
date: Sun, 02 Aug 2026 18:18:35 GMT
server: RestErl
```

Schemas:

- CreateMenuItemDto >
- MenuItemDto >

- Screenshot of the GET Request for the already Deleted Dish (Status Code: 404)

The screenshot displays the Swagger UI for the CampusEats.Api v1. The interface is in a dark theme. The top navigation bar shows the Swagger logo and the API version. The main content area is titled "Menu" and lists several endpoints. The selected endpoint is "GET /api/Menu/{id}". The parameters section shows a required integer parameter "id" with the value "4". The "Execute" button is highlighted. Below the "Execute" button, the "Responses" section is expanded, showing a 404 status code with the message "Error: Not Found". The response body is displayed in a dark box, showing a JSON object with the following structure:

```
{  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.3",  "title": "Not Found",  "status": 404,  "traceId": "90-f7612547b6fc/escf37880f09f28359-72e3b6d195b304-90"}
```

 The response headers are also visible, showing "Content-Type: application/problem+json; charset=utf-8", "Date: Sun, 02 Aug 2020 18:19:48 GMT", "Server: Restful", and "Transfer-Encoding: chunked". At the bottom, the "Schemas" section is expanded, showing the "CreateMenuItemDto" and "MenuItemDto" schemas.

- Screenshot of the POST Request of Creating a Dish leaving the Name field empty (Status Code: 400)

The screenshot displays the Swagger UI interface for the CampusEats.Api. The 'POST /api/Menu' endpoint is selected. The request body is set to application/json and contains the following JSON:

```
{  "name": "",  "price": 300,  "category": "Drinks"}
```

The 'Execute' button has been clicked, resulting in a 400 Bad Request status code. The response body is as follows:

```
{  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",  "title": "One or more validation errors occurred.",  "status": 400,  "errors": {    "name": [      "The Name field is required."    ]  },  "traceId": "90-65b73ccfd6d773d8bc5b0953a83cd-1f67cb77b08c9f-00"}
```

The response headers are:

```
content-type: application/problem+json; charset=utf-8date: Sun, 02 Aug 2026 18:21:36 GMTserver: Kestreltransfer-encoding: chunked
```

The 'Responses' section shows a 200 OK status code with a media type of text/plain. The 'Schemas' section lists the CreateMenuDto and MenuDto schemas.

Part B – Justify your design (LO4)

Operation	Method	Path	Success Status + Why
List all dishes	GET	/api/menu	Status : 200 OK Reason : The request is successful and returns the list of all menu items.
Get one dish by id	GET	/api/menu/{id}	Status : 200 OK Reason : Returns the requested menu item if it exists.
Add a new dish	POST	/api/menu	Status : 201 Created Reason : A new resource is created, and its details (including the generated ID) are returned.
Replace a dish	PUT	/api/menu/{id}	Status : 204 No Content Reason : The existing resource is successfully updated, and no response body is needed.
Remove a dish	DELETE	/api/menu/{id}	Status : 204 No Content Reason : The resource is successfully deleted, and no response body is required.

Short Justification for ASP.NET Core

- ASP.NET Core is a suitable framework for the CampusEats backend because it has high-performance and cross-platform supported, allowing applications to run efficiently on Windows, Linux, and macOS while supporting modern deployment methods such as containers. It also provides built-in features such as dependency injection, model validation, routing, and Swagger integration, making it easier to develop, test, and maintain RESTful APIs. Additionally, .NET 8 is a Long-Term Support (LTS) release, making it a reliable choice for enterprise applications.

Quick Test – Knowledge Check

Multiple Choice (Choose One)

Q1. (c) decimal

Q2. (b) 400 Bad Request on an invalid model

Q3. (b) constructor injection (DI)

Q4. (b) 201 Created

Q5. (b) to hide internal fields and decouple the API from the model

REST Design

Q6.

- **HTTP Method** : GET
- **Path** : /api/menu/{id}
- **Success Status Code** : 200 OK
- **Status Code when the dish does not exist** : 404 Not Found

Code-based

Q7.

```
[HttpGet("{id}")]
public ActionResult GetById(int id)
{
    var item = _svc.GetById(id);
    return item is null ? NotFound() : Ok(item);
}
```

Scenario-based

Q8.

- **Reason why PUT responded with 400 Bad Request,**
 - PUT is designed to replace the entire resource, so the request body must include all required fields defined by the resource or DTO. Since the request only contains the price field, the required name and category fields are missing, causing model validation to fail and ASP.NET Core to return 400 Bad Request.
- **HTTP method which is designed to use for partial updates,**
 - The HTTP method designed for partial updates is **PATCH**, which allows updating only the specified fields without replacing the entire resource.